

An Empirical Evaluation of LLMs for Microservice Bad Smell Detection

Mateus Dutra

Agenda

- Introduction
- Background
- Study Design
- Expected Results
- Limitations
- Conclusion

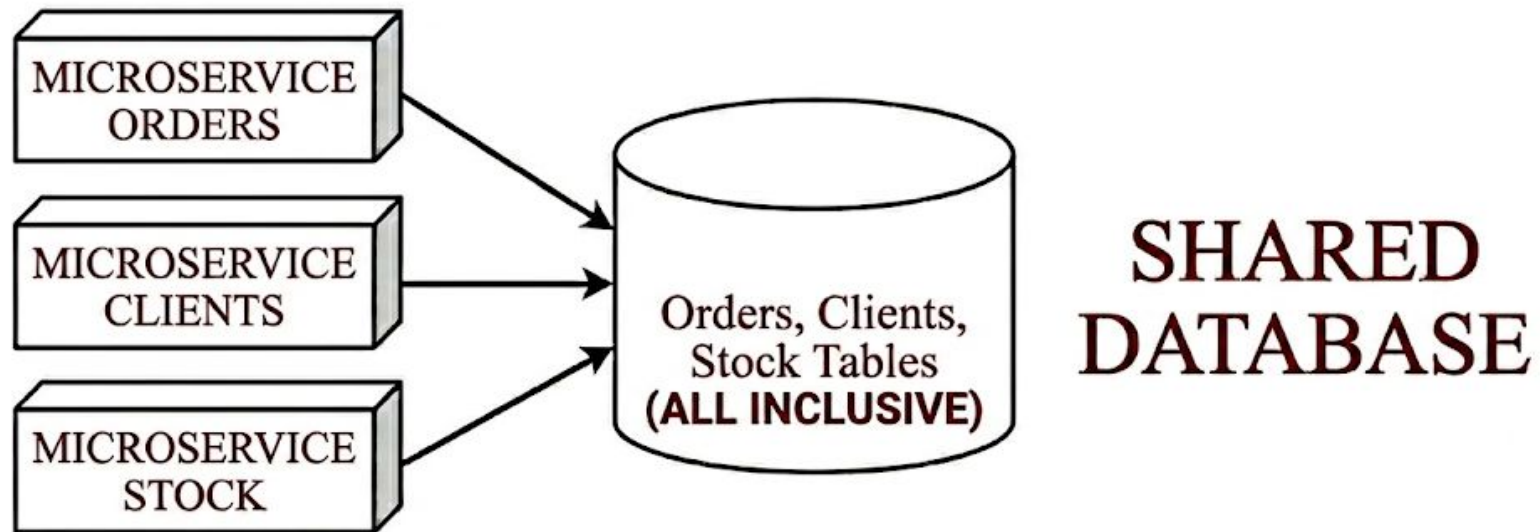
Introduction

- ❑ Microservices are widely adopted by the industry
- ❑ Systems degrades over time due to poor design decisions
- ❑ Detection is:
 - Costly
 - Limited
- ❑ Opportunity: use LLMs 

Background

- Microservices Bad Smells (MBS) are symptoms of poor design
- Examples:
 - No API Gateway
 - Shared Database
 - Cyclic Dependency
 - Shared Libraries

Background



PROBLEM: Various services access and directly modify the same database.

CONSEQUENCE: Tight coupling, scalability difficulties, schema dependency.

Background

- Previous study:
 - Systematic Literature Review on MBS
 - 104 smells
 - 7 detection tools
 - 15 refactoring strategies for the most discussed ones

Research Questions

- **RQ1:** How effective are LLMs at detecting MBS?
- **RQ2:** How does detection performance vary across different models?
- **RQ3:** How does detection performance vary across different MBS?

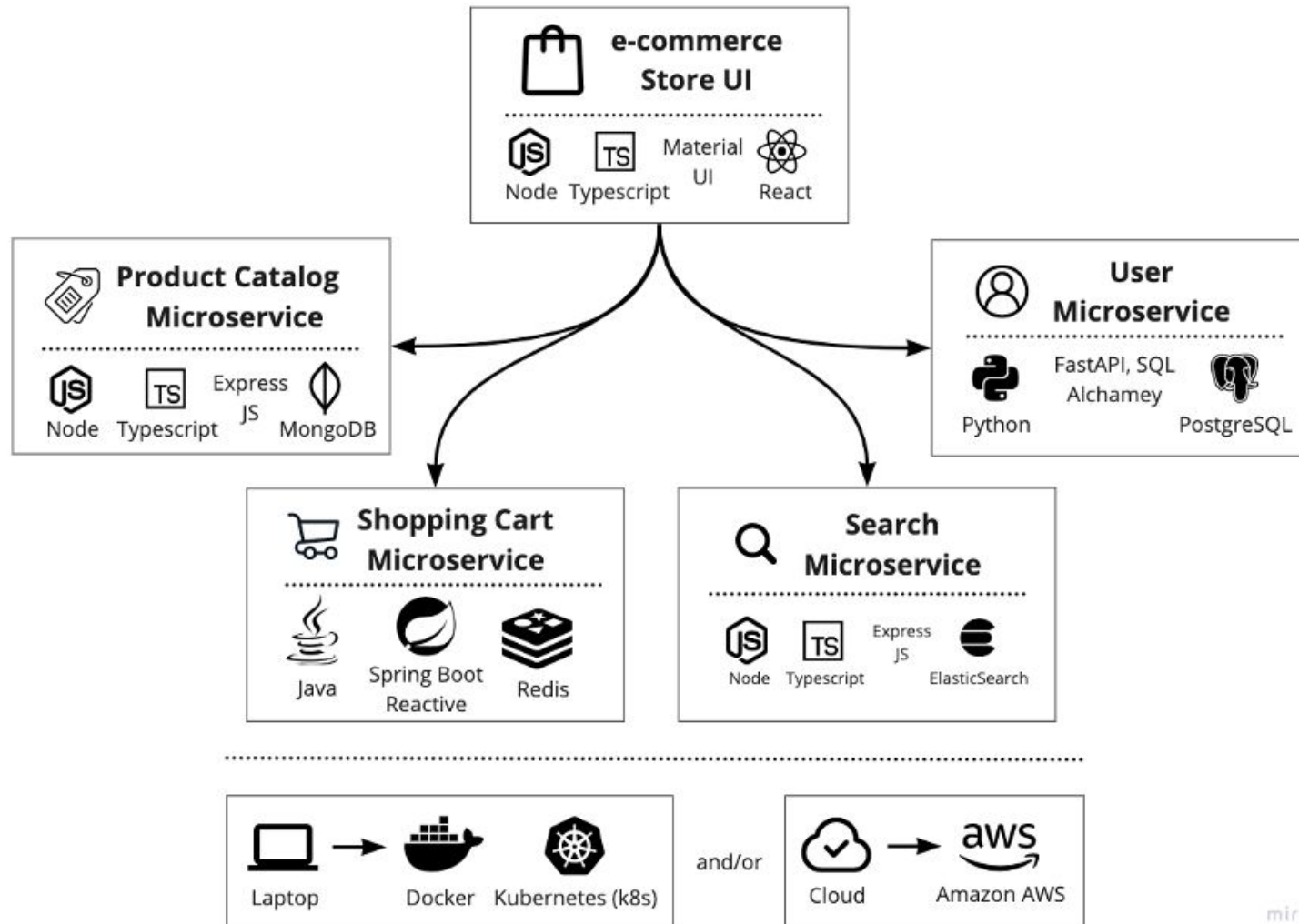
Study Design

- Smell selection
 - Focus on specific smells
 - No API Gateway
 - Nano Service
 - Shared Database
 - Microservice Greedy
 - Cyclic Dependency

Study Design

- Models selection
 - Coding-specific models
 - Claude Opus 4.6
 - GPT-5.4
 - Deepseek-coder-v2
 - Qwen-coder-plus
- Dataset
 - Filtered architecture diagrams from an open-source microservices dataset

Study Design



Study Design

- Prompting strategy
 - Clear definitions
 - Input the diagram as an image
 - Structured output format

```
{  
  "smell": "Nano Service",  
  "confidence": number,  
  "involved_services": string,  
  "explanation": string  
}
```

Study Design

You are a ****senior software architect**** analyzing microservice architecture diagrams to detect bad smells.

The smell to detect is ****CYCLIC DEPENDENCY****: two or more services form a circular dependency chain (e.g., $A \rightarrow B \rightarrow C \rightarrow A$), making independent deployment impossible for the involved services and complicating fault isolation.

Considering the attached diagram, what is your confidence on the presence of the Cyclic Dependency smell?

- 1 - Not present
- 2 - Unlikely present
- 3 - Likely present
- 4 - Clearly present

Respond only with a JSON object following this schema:

```
{  
  "smell": "Cyclic Dependency",  
  "confidence": number,  
  "involved_services": [string],  
  "explanation": string  
}
```

← Role

← Smell
description

← Confidence

← Output
format

Study Design

- Ground truth construction
 - Online questionnaire to validate the presence of selected smells in architectural diagrams
 - Confirmation criteria:
 - A smell will be confirmed if identified by at least two participants
 - Participants:
 - Software engineering researchers
 - Level of agreement will be assessed to evaluate reliability

Pilot study

- Goal: validate the experimental setup
- Setup:
 - Small subset of architectures
 - Multiple runs per model
- Findings
 - The model performed better separating prompts per smell
 - Very small variability across runs
- Improvements
 - Separate prompts

Execution

- Input: architecture + prompt
- LLM predicts smells
- Output parsed into structured format
- Compare with ground truth
- Multiple runs per configuration
- Metrics:
 - Precision
 - Recall
 - F1-score

Expected Results

- Empirical evaluation of LLMs
- Comparison across models
- Insights on MBS detection
- Contribution:
 - Understanding LLMs in architecture analysis

Conclusion

- Investigating LLMs for MBS and Software Architecture
- Ongoing experiment

Thank you!
Questions?